

1. Write a C program to implement the following operations on a 1D Array

a) Insert b) Delete c) Search d) Traverse

Code:

// Program to perform INS, DEL, SEARCH, TRAVERSE operations on an array

```
#include <stdio.h>
#define SIZE 50
void main()
{
    int arr[SIZE];
    int n, choice, i, pos, value, found;
    printf("\nEnter the initial size of the array: ");
    scanf("%d", &n);
    printf("\nEnter elements into the array:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    do
    {
        printf("\n---1D ARRAY OPERATIONS---\n");
        printf("1.      Insert\n");
        printf("2.      Delete\n");
        printf("3.      Search\n");
        printf("4.      Traverse\n");
        printf("5.      Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice)
        {
            case 1:
                if(n == SIZE)
                {
                    printf("Array is full\n");
                    break;
                }
                printf("\nEnter position of insertion: ");
                scanf("%d", &pos);
                printf("Enter element to be inserted: ");
                scanf("%d", &value);
                for(i = n; i >= pos; i--)
                {
                    arr[i] = arr[i - 1];
                }
                arr[pos - 1] = value;
                n++;
                printf("Array after insertion is:\n");
                for(i = 0; i < n; i++)
                {
                    printf("%d\n", arr[i]);
                }
            }
        }
    }
```

```

    }
    break;
case 2:
    if(n == 0)
    {
        printf("Array is empty\n");
        break;
    }
    printf("\nEnter position of deletion: ");
    scanf("%d", &pos);
    for(i = pos - 1; i < n; i++)
    {
        arr[i] = arr[i + 1];
    }
    n--;
    printf("\nArray after deletion is:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d\n", arr[i]);
    }
    break;
case 3:
if(n == 0)
    {
        printf("Array is empty\n");
        break;
    }
    printf("Enter the value to search: ");
    scanf("%d", &value);
    found = 0;
    for(i = 0; i < n; i++)
    {
        if(arr[i] == value)
        {
            printf("Element is found at position %d\n", i + 1);
            found = 1;
            break;
        }
    }
    if(found == 0)
    {
        printf("Element not found\n");
    }
    break;
case 4:
    if(n == 0)
    {
        printf("Array is empty\n");
        break;
    }

```

```
    printf("Array elements are:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d\n", arr[i]);
    }
    break;
case 5:
    printf("Exiting...\n");
    break;
default:
    printf("Invalid choice\n");
}
}
while(choice != 5);
}
```

2. Write a program to implement Self-Referential Structure.

Code:

```
// Program to implement self referential structure
#include <stdio.h>
#include <stdlib.h>
struct node
{
    int data;
    struct node *link;
};
void main()
{
    struct node *n1, *n2, *n3;
    // Dynamic memory allocation to nodes
    n1 = (struct node *)malloc(sizeof(struct node));
    n2 = (struct node *)malloc(sizeof(struct node));
    n3 = (struct node *)malloc(sizeof(struct node));
    // Create three nodes
    n1->data = 100;
    n1->link = NULL
    n2->data = 200;
    n2->link = NULL;
    n3->data = 300;
    n3->link = NULL;
    // Link nodes
    n1->link = n2;
    n2->link = n3;
    // Print the values of nodes
    printf("First node: %d\n", n1->data);
    printf("Second node: %d\n", n1->link->data);
    printf("Third node: %d\n", n2->link->data);
}
```

3. Write a C program to perform Dynamic Memory Allocation.

Code:

```
// Program for Dynamic Memory Allocation
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int *ptr, n, i, newsize;
    printf("Enter no. of elements: ");
    scanf("%d", &n);
    ptr = (int *)malloc(n * sizeof(int));
    if(ptr == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }
    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &ptr[i]);
    }
    printf("Elements using malloc:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", ptr[i]);
    }
    printf("\nEnter new size: ");
    scanf("%d", &newsize);
    ptr = (int *)realloc(ptr, newsize * sizeof(int));
    printf("Enter remaining elements:\n");
    for(i = n; i < newsize; i++)
    {
        scanf("%d", &ptr[i]);
    }
    printf("Elements after reallocation:\n");
    for(i = 0; i < newsize; i++)
    {
        printf("%d ", ptr[i]);
    }
    free(ptr);
    return 0;
}
```

4. Write a program to implement Singly Linked List with the following operations:

i) Creation ii) Insertion iii) Deletion iv) Search v) Display

Code:

```
//Program to implement linked list operations
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct node
```

```
{
```

```
    int data;
```

```
    struct node*next;
```

```
};
```

```
struct node *ptr,* head,*temp,*temp1;
```

```
int val;
```

```
void insertbeg()
```

```
{
```

```
    ptr=(struct node*)malloc(sizeof(struct node));
```

```
    printf("\n enter the value");
```

```
    scanf("%d",&val);
```

```
    ptr->data=val;
```

```
    ptr->next=NULL;
```

```
    if(head==NULL)
```

```
    {
```

```
        head=ptr;
```

```
    }
```

```
    else
```

```
    {
```

```
        ptr->next=head;
```

```
        head=ptr;
```

```
    }
```

```
}
```

```
void insertend()
```

```
{
```

```
    ptr=(struct node*)malloc(sizeof(struct node));
```

```
    printf("enter a value");
```

```
    scanf("%d",&val);
```

```
    ptr->data=val;
```

```
    ptr->next=NULL;
```

```
    if(head==NULL)
```

```
    {
```

```
        head=ptr;
```

```
    }
```

```
    else
```

```
    {
```

```
        temp=head;
```

```
        while(temp!=NULL)
```

```
        {
```

```
            temp=temp->next;
```

```
        }
```

```
        temp->next=ptr;
```

}

25H51A6707

```

void insertany()
{
    ptr=(struct node*)malloc(sizeof(struct node));
    printf("enter a value");
    scanf("%d",&val);
    ptr->data=val;
    ptr->next=NULL;
    int pos,i=0;
    printf("\n enter the position");
    scanf("%d",&pos);
    if(head==NULL&&pos==0)
    {
        head=ptr;
    }
    else
    {
        temp=head;
        while(i<pos-1)
        {
            temp1=temp;
            temp=temp->next;
            i++;
        }
        ptr->next=temp;
        temp1->next=ptr;
    }
}

```

```

void traverse()
{
    if(head==NULL)
        printf("empty list");
    else
    {
        temp=head;
        while(temp->next!=NULL)
        {
            printf("%d->",temp->data);
            temp=temp->next;
        }
        printf("NULL");
    }
}

```

```

void deletebeg()
{
    if(head==NULL)
        printf("empty list");
    else
    {
        temp=head;

```

```
head=head->next;  
temp->next=NULL;
```

25H51A6707

```

        free(temp);
    }
}

void deleteend()
{
    if(head==NULL)
        printf("empty list");
    else
    {
        if(head==NULL)
            printf("\n Empty list");
        else
        {
            temp=head;
            while(temp->next!=NULL)
            {
                temp1=temp;
                temp=temp->next;}
            temp1->next=NULL;
            free(temp);
        }
    }
}

void deleteany()
{
    if(head==NULL)
        printf("empty list");
    else
    {
        temp=head;
        int i=0,pos;
        printf("\n enter position to delete");
        scanf("%d",&pos);
        while(1<pos-1)
        {
            temp1=temp;
            temp=temp->next;
        }
        temp1->next=temp->next;
        temp->next=NULL;
        free(temp);
    }
}

void search()
{
    if(head==NULL)
        printf("empty list");
    else
    {

```

```
int key,found=1,x=0;  
printf("\n enter search element");
```

25H51A6707

```

scanf("%d",&key);
temp=head;
while(temp!=NULL)
{
    if(temp->data==key)
    {
        printf("\n element found at %d",found);
        x=1;
    }
    temp=temp->next;
    found++;
}
if(x==0)
    printf("\n elemnt not found");
}

}

void main()
{
    int choice;
    while(1)
    {
        printf("\n SINGLE LINKED LIST OPERATIONS:");
        printf("\n 1 Insert begining\n 2 Insert end\n");
        printf("3 Insert specific position\n 4 Display\n");
        printf("5 Exit\n 6 Delete begining\n 7 Delete end\n");
        printf("8 Delete any\n 9 Search\n");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1: insertbeg();break;
            case 2: insertend();break;
            case 3: insertany();break;
            case 4: traverse();break;
            case 5: exit(0);break;
            case 6: deletebeg();break;
            case 7: deleteend();break;
            case 8: deleteany();break;
            case 9: search();break;
            default:("invalid choice");
        }
    }
}

```

5. Write a function to reverse the nodes of a singly linked list.

Code:

```
//program to reverse the single linked list
#include<stdio.h>
#include<stdlib.h>
#define max 10
int stk[max];
int top=-1;
struct node
{
int data;
struct node *next;
}*first=NULL;
void create()
{
struct node *newnode,*temp;
int item;
while(1)
{
printf("\nEnter data:");
scanf("%d",&item);
if(item== -77)
return;
newnode=(struct node *)malloc(sizeof(struct node));
newnode->data=item;
newnode->next=NULL;
if(first==NULL)
{ first=newnode;temp=newnode; }
else
{ temp->next=newnode;temp=newnode;}
}
}
void display()
{
struct node *temp;
if(first==NULL)
printf("List is empty!!");
else
{
temp=first;
while(temp!=NULL)
{
printf("%d->",temp->data);
temp=temp->next;
}printf("NULL");
}
}
void
reverse(){ struct
node *temp;
temp=first;
```

```
while(temp!=NULL)
{
top=top+1;
stk[top]=temp->data;
temp=temp->next;
}
temp=first;
while(top!=-1)
{
temp->data=stk[top];
top=top-1;
temp=temp->next;
}
}
void main()
{
create();
printf("\nLinked list is :\n");
display();
printf("\nReversed Linked list is :\n");
reverse();
display();
}
```

6. Write a program that implements Stack (its operations) using Arrays

Code:

```
// Program to implement Stack operations using Arrays
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int top=-1,max,a[100];
```

```
void push();
```

```
void pop();
```

```
void display();
```

```
int isempty();
```

```
int isfull();
```

```
int main()
```

```
{
```

```
    int ch;
```

```
    printf("Enter the size of stack\n");
```

```
    scanf("%d",&max);
```

```
    printf("\nOperations performed by Stack");
```

```
    printf("\n1.Push the elements \n 2.Pop the elements \n
```

```
        3.Display \n 4.End \n");
```

```
    do
```

```
    {
```

```
        printf("Enter the choice\n");
```

```
        scanf("%d",&ch);
```

```
        switch(ch)
```

```
        {
```

```
            case 1:push();break;
```

```
            case 2:pop();break;
```

```
            case 3:display();break;
```

```
            case 4:exit(0);break;
```

```
            default:printf("Invalid choice!");
```

```
        }
```

```
    }while(ch<4);
```

```
}
```

```
void push()
```

```
{
```

```
    int ele;
```

```
    if(top==max-1)
```

```
    {
```

```
        printf("\n Stack Over flow!");
```

```
    }
```

```
    else
```

```
    {
```

```
printf("\n Enter elementsto be inserted to the stack");  
scanf("%d",&ele);
```

25H51A6707

```

    top++;
    a[top]=ele;
}
}
void pop()
{
    int ele;
    if(top== -1)
    {
        printf("\n Stack Under flow!");
    }
    else
    {
        ele=a[top];
        top--;
        printf("\n Deleted element is=%d\n",ele);
    }
}
void display()
{
    int i,x;
    if(top== -1)
    {
        printf("Stack is Empty!");
    }
    else
    {
        printf("\nElements present in the stack");
        for(i=0;i<=top;i++)
            printf("%d\t",a[i]);
    }
}
int isempty()
{
    if(top== -1)
        return 1;
    else
        return 0;
}
int isfull()
{
    if(top==max-1)
        return 1;

```

```
else  
    return 0;}
```

25H51A6707

7. Write a program that implements Queue (its operations) using Arrays

Code:

```
//program to implement queue using arrays
#include<stdio.h>
#include<stdlib.h>
int front=0,rear=0,que[100],max;
void enqueue();
void dequeue();
void display();
int ele;
int isFull()
{
    if(rear==max-1)
        return 1;
    else
        return 0;
}
int isEmpty()
{
    if(front==rear)
        return 1;
    else
        return 0;
}
int main()
{
    int ch;char op;
    printf("enter the size of Q:\n");
    scanf("%d",&max);
    printf("\nOperations performed on queue");
    printf("\n1.Enqueue\n2.Dequeue\n3.Display\n4.End");
    do
    {
        printf("\nEnter the choice:");
        scanf("%d",&ch); switch(ch)
        {
            case 1:enqueue();break;
            case 2:dequeue();break;
            case 3:display();break;
            case 4:exit(0);
            default:printf("\nInvalid choice!!");
        }
    }while(ch<4);
    return 0;
}
void enqueue()
{
    int ele;
    if(isFull())
```

```

    {
        printf("\nOverflow!!");
    }
    else
    {
        printf("\nEnter element to be inserted");
        scanf("%d",&ele);
        que[rear]=ele;
        rear++;
    }
}
void deque()
{
    int      ele;
    if(isEmpty())
    {
        printf("\nQueue Underflow!!");
    }
    else
    {
        ele=que[front];
        front++;
        printf("\ndeleted element is=\n%d",ele);
    }
}
void display()
{
    int  i;
    int ele;
    if(isEmpty())
    {
        printf("\nQueue is empty");
    }
    else
    {
        printf("\nElements present in th Q:\n");
        for(i=front;i<rear;i++)
            printf("%d\t",que[i]);
    }
}
}

```

8. Write a program that implements Circular Queue (its operations) using Arrays

Code:

```
//program to implement circular queue
#include<stdio.h>
#include<stdlib.h>
#define max 5
int f=-1,r=-1,cq[max];
void insert();
void delete();
void display();
void main()
{
    int ch;
    do
    {
        printf("\nCircular Queue Operations\n");
        printf("\n1.Insertion\n2.Deletion\n3.Display\n4.Exit\n");
        printf("\nEnter choice");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:insert();break;
            case 2:delete();break;
            case 3:display();break;
            case 4:exit(0);
            default:printf("\nInvalid choice!!");
        }
    }while(ch<4);
}
void insert()
{
    int item;
    if(f==0&&r==(max-1))
    {
        printf("Queue Full!!");
    }
    else
    {
        printf("\nEnter element");
        scanf("%d",&item);
        if(f==-1&&r==-1)
        {
            f=0;r=0;cq[r]=item;
        }
        else if(r==(max-1))
        {
            r=0;cq[r]=item;
        }
    }
    else
    {

```

```

        r++;cq[r]=item;
    }
}
}
void delete()
{
    int item;
    if(f== -1 && r== -1)
    {
        printf("Queue Empty!!");
    }
    else
    {
        if(f==r)
        {
            item=cq[r];
            f=-1;
            r=-1;
            printf("Deleted element=%d",item);
        }
        else if(f==(max-1))
        {
            item=cq[r];
            f=0;
            printf("Deleted elements=%d",item);
        }
        else
        {
            item=cq[f];
            f++;
            printf("Deleted elements=%d",item);
        }
    }
}
void display()
{
    int i;
    if(f== -1 && r== -1)
        printf("Queue Empty!!");
    else
    {
        if(f<r)
        {
            for(i=f;i<=r;i++)
            {
                printf("%d",cq[i]);
            }
        }
        else
        {

```

```
    for(i=f;i<=max-1;i++)  
        printf("%d",cq[i]);  
    for(i=0;i<=r;i++)  
        printf("%d",cq[i]);  
    }  
    }  
}
```

25H51A6707

9. Write a program that implements Stack (its operations) using Linked List.

Code:

```
//program to implement stack using linked list
#include<stdio.h>
#include<stdlib.h>
typedef struct list
{
    int data;
    struct list *link;
}node;
node *top=NULL;
void push();
void pop();
void peek();
void display();
int main()
{
    int ch;
    char op;
    do
    {
        printf("\n Menu\n1.Push\n2.Pop\n3.Display\n4.Peek\n5.Exit\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:push();break;
            case 2:pop();break;
            case 3:display();break;
            case 4:peek();break;
            case 5:exit(0);
            default:printf("Invalid choice\n");
        }
    }while(ch<5);
}
void push()
{
    node *newnode;
    int item;
    printf("Enter an item for insertion\n");
    scanf("%d",&item);
    newnode=(node*)malloc(sizeof(node));
    newnode->data=item;
```

```
newnode->link=top;  
top=newnode;
```

25H51A6707

```

}
void pop()
{
    node *temp;
    if(top==NULL)
    {
        printf("Stack is empty!");
    }
    else
    {
        printf("Deleted item=%d",top->data);
        temp=top;
        top=top->link;
        free(temp);
    }
}
void peek()
{
    if(top==NULL)
    {
        printf("Stack is empty!!");
    }
    else
    {
        printf("The top most element in the stack is %d",top->data);
    }
}
void display()
{
    node *temp=top;
    if(top==NULL)
    {
        printf("Stack is empty");
    }
    else
    {
        printf("Stack is\n");
        printf("START->");
        while(temp!=NULL)
        {
            printf("%d->",temp->data);
            temp=temp->link;
        }
    }
}

```

```
printf("NULL");  
}}
```

25H51A6707

10. Write a program that implements Queue (its operations) using Linked List.

Code:

//Program to implement Queue using linked list.

```
#include<stdio.h>
#include<stdlib.h>
typedef struct queue
{
    int data;
    struct queue *link;
}node;
node *front=NULL,*rear=NULL;
void insertion();
void deletion();
void display();
int main()
{
    int ch;
    char op;
    do
    {
        printf("\nMENU\n");
        printf("1.Insert \n 2.Delete\n 3.Display\n 4.exit\n");
        printf("\nEnter your choice\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:insertion();break;
            case 2:deletion();break;
            case 3:display();break;
            case 4:exit(0);
            default:printf("\nInvalid choice\n");
        }
    }while(ch<4);
    return 0;
}
void insertion()
{
    node *newnode;
    int item;
    printf("\nEnter an item for insertion \n");
    scanf("%d",&item);
    newnode=(node*)malloc(sizeof(node));
```

```
newnode->data=item;  
newnode->link=NULL;
```

25H51A6707

```

if(front==NULL && rear==NULL)
{
    front=newnode;
    rear=newnode;
}
else
{
    rear->link=newnode;
    rear=newnode;
}
}
void deletion()
{
    node *temp;
    if(front==NULL)
    {
        printf("\nQueue is empty\n");
    }
    else
    {
        printf("\nDeleted item=%d\n",front->data);
        temp=front;
        front=front->link;
        free(temp);
        if(front==NULL)
        {
            rear=NULL;
        }
    }
}
void display()
{
    node *temp=front;
    if(front==NULL)
    { printf("\nQueue is empty\n");}
    else
    {printf("FRONT->");
        while(temp!=NULL)
        {
            printf("%d->",temp->data);
            temp=temp->link;
        }
        printf("REAR\n");
    }
}

```

}

}

25H51A6707

11. Write a C program to Convert the given Infix Expression to Postfix Expression.

Code:

```
// program to convert infix expression to postfix expression
#include<stdio.h>
#include<string.h>
#include<ctype.h>
#define max 30
char stack[max];
int top = -1;
void push(char item)
{
    top = top + 1;
    stack[top] = item;
}
char pop()
{
    char item;
    item = stack[top];
    top = top - 1;
    return item;
}

int prec(char c)
{
    if(c == '(') return 0;
    if(c == '+' || c == '-') return 1;
    if(c == '*' || c == '/' || c == '%') return 2;
    if(c == '^') return 3;
}

void postfix()
{
    char infix[30], p[30], ch;
    int i, j = 0;

    push('(');

    printf("\nEnter the infix expression: ");
    scanf("%s", infix);

    for(i = 0; infix[i] != '\0'; i++)
    {
        if(isalpha(infix[i]))
        {
            p[j] = infix[i];
            j++;
        }
    }
}
```

```

    }

    if(infix[i]=='+' || infix[i]=='-' || infix[i]=='*' || infix[i]=='/' || infix[i]=='%' ||
infix[i]=='^')
    {
        while(prec(stack[top]) >= prec(infix[i]))
        {
            p[j] = pop();
            j++;
        }
        push(infix[i]);
    }
}

```

```

if(infix[i] == '(')
push(infix[i]);

```

```

    if(infix[i] == ')')
    {
        while(stack[top] != '(')
        {
            p[j] = pop();
            j++;
        }
        ch = pop();
    }
}

```

```

p[j] = '\0';
puts(p);
}

```

```

void main()
{
    postfix();
    postfix();
}

```

12. Write a C program to Evaluate the given Postfix Expression.

Code:

```
//program to evaluate postfix expression
#include<stdio.h>
#include<string.h>
#include<math.h>
#include<ctype.h>
#define max 30
void posteval();
char stack[max];
int top=-1;
void push(char item)
{
    top=top+1;
    stack[top]=item;
}
char pop()
{
    char item;
    item=stack[top];
    top=top-1;
    return item;
}
void posteval()
{
    char p[30];
    int i, opd, a, b;
    printf("\nEnter arithmetic expression in postfix notation ending with ')': ");
    scanf("%s", p);
    for(i=0; p[i]!=')'; i++)
    {
        if(isalpha(p[i]))
        {
            printf("Enter the value of %c", p[i]);
            scanf("%d", &opd);
            push(opd);
        }
        else
        {
            a=pop();
            b=pop();
            switch(p[i])
            {
                case '+': push(b+a); break;
                case '-': push(b-a); break;
                case '*': push(b*a); break;
                case '/': push(b/a); break;
                case '%': push(b%a); break;
                case '^': push(pow(b,a)); break;
            }
        }
    }
}
```

```
    }  
    }  
    printf("\nThe value of postfix expression=%d\n",stack[top]);  
}  
void main()  
{  
    posteval();  
}
```

25H51A6707